

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 4**



Sorting

Oleh:

Muhammad Kurniawan Pasya

NIM. 2510817210026

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 4

Laporan Praktikum Algoritma & Struktur Data Modul 4: Sorting, ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Kurniawan Pasya

NIM : 2510817210026

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL.....	6
A. Source Code.....	7
B. Pembahasan dan Output Program.....	13
BUBBLE SORT	13
SELECTION SORT	17
INSERTION SORT.....	20
MERGE SORT	23
QUICK SORT	26
RADIX SORT	29
TAUTAN GIT	32

DAFTAR GAMBAR

Gambar 1 Screenshot Hasil Uji Coba Bubble Sort	14
Gambar 2 Screenshot Hasil Uji Coba Selection Sort	17
Gambar 3 Screenshot Hasil Uji Coba Insertion Sort.....	20
Gambar 4 Screenshot Hasil Uji Coba Merge Sort.....	23
Gambar 5 Screenshot Hasil Uji Coba Quick Sort	26
Gambar 6 Screenshot Hasil Uji Coba Radix Sort	29

DAFTAR TABEL

Tabel 1 Source Code sorting_algorithms.cpp.....	7
---	---

SOAL

Algoritma Sorting

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Radix Sort

Instruksi Pengujian

Lakukan pengujian terhadap setiap algoritma menggunakan beberapa kondisi data berikut:

1. Data acak (**random**).
2. Data sudah (atau hampir) terurut menaik.
3. Data terurut menurun.
4. Data dengan banyak elemen duplikat.

Hal yang Perlu Dianalisis

Untuk setiap algoritma, lakukan analisis terhadap aspek berikut:

1. Kompleksitas waktu:
 - Best Case
 - Average Case
 - Worst Case
2. Karakteristik algoritma:
 - Apakah algoritma bersifat stable atau tidak.
 - Apakah algoritma termasuk in-place atau membutuhkan memori tambahan.
 - Mekanisme utama algoritma dalam melakukan pengurutan.

3. Analisis operasi:

- Jumlah comparison.
- Jumlah swap.
- Jumlah shift (khusus algoritma tertentu seperti Insertion Sort).
- Jumlah recursion call.
- Jumlah array accessed.
- Pengaruh bentuk data terhadap jumlah operasi.

4. Analisis performa:

- Perbandingan waktu eksekusi antar algoritma.
- Algoritma mana yang paling cepat untuk dataset kecil dan besar.
- Pengaruh distribusi data terhadap performa algoritma.
- Hubungan antara teori kompleksitas dan hasil eksperimen yang diperoleh.

A. Source Code

Tabel 1 Source Code sorting_algorithms.cpp

1	#include "sorting_algorithms.h"
2	#include <algorithm>
3	#include <chrono>
4	
5	using Clock = std::chrono::high_resolution_clock;
6	
7	void bubble_sort(std::vector<int>& data, Metrics& m) {
8	int n = data.size();
9	auto start = Clock::now();
10	
11	bool swapped;
12	for (int i = 0; i < n - 1; ++i) {
13	swapped = false;
14	
15	for (int j = 0; j < n - i - 1; ++j) {
16	m.comparisons++;

```
17
18         if (data[j] > data[j + 1]) {
19             std::swap(data[j], data[j + 1]);
20             m.swaps++;
21             swapped = true;
22         }
23     }
24     if (!swapped) break;
25 }
26
27     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
28 }
29
30
31
32 void selection_sort(std::vector<int>& data, Metrics& m)
33 {
34     int n = data.size();
35     auto start = Clock::now();
36
37     for (int i = 0; i < n - 1; ++i) {
38         int min_idx = i;
39
40         for (int j = i + 1; j < n; ++j) {
41             m.comparisons++;
42
43             if (data[j] < data[min_idx]) {
44                 min_idx = j;
45             }
46         }
47         if (min_idx != i) {
48             std::swap(data[i], data[min_idx]);
49             m.swaps++;
50         }
51     }
52
53     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
54 }
55
56
57
58
```



```

59 void insertion_sort(std::vector<int>& data, Metrics& m)
60 {
61     int n = data.size();
62     auto start = Clock::now();
63
64     for (int i = 1; i < n; ++i) {
65         int key = data[i];
66         int j = i - 1;
67
68         while (j >= 0 && data[j] > key) {
69             m.comparisons++;
70             data[j + 1] = data[j];
71             m.shifts++;
72             j--;
73         }
74         if (j >= 0) m.comparisons++;
75         data[j + 1] = key;
76         m.shifts++;
77     }
78     m.time_ms = std::chrono::duration<double,
79 std::milli>(Clock::now() - start).count();
80 }
81
82
83 void do_merge(std::vector<int>& data, int left, int
84 mid, int right, Metrics& m, std::vector<int>& temp) {
85     int i = left, j = mid + 1, k = left;
86
87     while (i <= mid && j <= right) {
88         m.comparisons++;
89         if (data[i] <= data[j]) {
90             temp[k++] = data[i++];
91             m.array_accesses += 2;
92         } else {
93             temp[k++] = data[j++];
94             m.array_accesses += 2;
95         }
96     }
97     while (i <= mid) {
98         temp[k++] = data[i++];
99         m.array_accesses += 2;
100    }

```

```

101     while (j <= right) {
102         temp[k++] = data[j++];
103         m.array_accesses += 2;
104     }
105     for (i = left; i <= right; ++i) {
106         data[i] = temp[i];
107         m.array_accesses += 2;
108     }
109 }
110 void do_merge_sort(std::vector<int>& data, int left,
111 int right, Metrics& m, std::vector<int>& temp) {
112     m.recursive_calls++;
113
114     if (left < right) {
115
116         int mid = left + (right - left) / 2;
117
118         do_merge_sort(data, left, mid, m, temp);
119         do_merge_sort(data, mid + 1, right, m, temp);
120         do_merge(data, left, mid, right, m, temp);
121     }
122 }
123
124 void merge_sort(std::vector<int>& data, Metrics& m) {
125     auto start = Clock::now();
126
127     if (data.empty()) {
128         m.time_ms = 0.0;
129         return;
130     }
131
132     std::vector<int> temp(data.size());
133
134     do_merge_sort(data, 0, data.size() - 1, m, temp);
135
136     m.time_ms = std::chrono::duration<double,
137 std::milli>(Clock::now() - start).count();
138 }
139
140
141 void do_quick_sort(std::vector<int>& data, int low, int
142 high, Metrics& m) {

```

```
143     m.recursive_calls++;
144
145     if (low >= high) return;
146
147     int mid = low + (high - low) / 2;
148     int pivot = data[mid];
149     int i = low;
150     int j = high;
151
152     while (i <= j) {
153
154         while (true) {
155             m.comparisons++;
156             if (data[i] < pivot) i++;
157             else break;
158         }
159
160         while (true) {
161             m.comparisons++;
162             if (data[j] > pivot) j--;
163             else break;
164         }
165
166         if (i <= j) {
167             std::swap(data[i], data[j]);
168             m.swaps++;
169             i++;
170             j--;
171         }
172     }
173
174     do_quick_sort(data, low, j, m);
175     do_quick_sort(data, i, high, m);
176 }
177
178
179 void quick_sort(std::vector<int>& data, Metrics& m) {
180     auto start = Clock::now();
181
182     if (!data.empty()) {
183         do_quick_sort(data, 0, data.size() - 1, m);
184     }
185 }
```

```

186     m.time_ms = std::chrono::duration<double,
187 std::milli>(Clock::now() - start).count();
188 }
189
190
191
192 void count_sort_for_radix(std::vector<int>& data, int
193 exp, Metrics& m) {
194     int n = data.size();
195     std::vector<int> output(n);
196     std::vector<int> count(10, 0);
197
198     for (int i = 0; i < n; i++) {
199         int digit = (data[i] / exp) % 10;
200         count[digit]++;
201         m.array_accesses += 2;
202     }
203
204     for (int i = 1; i < 10; i++) {
205         count[i] += count[i - 1];
206         m.array_accesses += 3;
207     }
208
209     for (int i = n - 1; i >= 0; i--) {
210         int digit = (data[i] / exp) % 10;
211         output[count[digit] - 1] = data[i];
212         count[digit]--;
213         m.array_accesses += 4;
214     }
215
216     for (int i = 0; i < n; i++) {
217         data[i] = output[i];
218         m.array_accesses += 2;
219     }
220 }
221
222 void radix_sort(std::vector<int>& data, Metrics& m) {
223     if (data.empty()) return;
224     auto start = Clock::now();
225
226     int max_val = data[0];
227     for (int i = 1; i < data.size(); i++) {
228         if (data[i] > max_val) {
229             max_val = data[i];

```

230	}
231	}
232	for (int exp = 1; max_val / exp > 0; exp *= 10) { count_sort_for_radix(data, exp, m); }
	m.time_ms = std::chrono::duration<double, std::milli>(Clock::now() - start).count(); }

B. Pembahasan dan Output Program

BUBBLE SORT

Algoritma ini disusun menggunakan dua buah perulangan bersarang atau *nested loop*. Perulangan bagian luar berfungsi untuk memastikan seluruh elemen diperiksa, sedangkan perulangan bagian dalam bertugas membandingkan dua elemen yang posisinya bersebelahan. Jika elemen di sebelah kiri lebih besar daripada elemen di sebelah kanan, posisi keduanya akan ditukar atau swap. Tujuannya adalah agar nilai nilai yang besar akan mengapung seperti namanya bubble sort atau terdorong perlahan-lahan hingga mencapai ujung kanan array pada setiap akhir perputaran.

Selain itu, terdapat sebuah variabel penanda bernama `swapped`. Penanda ini sangat penting karena untuk mengecek apakah pada satu putaran penuh terjadi pertukaran. Jika tidak ada satu pun elemen yang ditukar, artinya seluruh data sudah berada di posisi yang benar, sehingga proses perulangan bisa langsung dihentikan lebih awal tanpa harus mengecek sampai akhir.

>> Bubble Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Bubble Sort	Random	100	4935	2564	0	0	0	0.030000
Bubble Sort	Sorted	100	99	0	0	0	0	0.001000
Bubble Sort	Reverse Sorted	100	4950	4950	0	0	0	0.022000
Bubble Sort	Nearly Sorted	100	4719	296	0	0	0	0.012000
Bubble Sort	Many Duplicates	100	4845	2315	0	0	0	0.023000
Bubble Sort	Random	1000	497154	243190	0	0	0	1.434000
Bubble Sort	Sorted	1000	999	0	0	0	0	0.002000
Bubble Sort	Reverse Sorted	1000	499500	499500	0	0	0	2.379000
Bubble Sort	Nearly Sorted	1000	481922	25778	0	0	0	0.741000
Bubble Sort	Many Duplicates	1000	496340	248808	0	0	0	1.656000
Bubble Sort	Random	10000	49993404	25118148	0	0	0	120.968000
Bubble Sort	Sorted	10000	9999	0	0	0	0	0.005000
Bubble Sort	Reverse Sorted	10000	49995000	49995000	0	0	0	119.648000
Bubble Sort	Nearly Sorted	10000	49937030	3119352	0	0	0	43.834000
Bubble Sort	Many Duplicates	10000	49993404	25093086	0	0	0	82.863000

Gambar 1 Screenshot Hasil Uji Coba Bubble Sort

Berdasarkan output tersebut, dapat dianalisis untuk **Kompleksitas Waktu**, **Best Case** nya terjadi apabila data sejak awal sudah terurut dari kecil ke besar. Saat program berjalan hanya akan membandingkan elemen-elemen bersebelahan satu putaran penuh dari awal hingga akhir. Karena posisinya sudah benar semua, tidak ada pertukaran yang terjadi. Variabel swapped bernilai false, dan perulangan langsung dihentikan. Kesimpulannya, kompleksitas waktu terbaiknya adalah $O(n)$.

Avarage Case terjadi ketika urutan elemen di dalam data acak. Program harus melakukan beberapa kali putaran dan banyak perbandingan serta pertukaran sebelum semua elemen berada pada posisi yang tepat. Secara rata-rata, kompleksitas waktunya adalah $O(n^2)$.

Worst Case terjadi ketika data awalnya terurut terbalik yaitu dari besar ke kecil. Algoritma harus menukar setiap elemen pada setiap langkah perbandingan di perulangan dalam. Tidak ada proses yang bisa dihentikan lebih awal, sehingga program bekerja secara maksimal. Kompleksitas waktu terburuknya adalah $O(n^2)$.

Untuk **Kompleksitas Ruang**, algoritma ini hanya memanipulasi posisi elemen langsung di dalam array bawaannya. Memori tambahan yang dipakai hanyalah beberapa variabel sederhana seperti untuk menyimpan status sementara saat proses menukar elemen, serta untuk penghitung perulangan. Kompleksitas ruangnya adalah $O(1)$, karena Bubble Sort bersifat in-place dan hanya memakai beberapa variabel tambahan

Untuk karakteristik algoritma ini ini bersifat Stable. Artinya, jika ada dua angka yang nilainya sama di dalam data asal, urutan asli mereka tidak akan berubah. Hal ini dikarenakan syarat pertukarannya mensyaratkan elemen kiri harus lebih besar dari elemen kanan. Jika nilainya

sama, posisinya tetap. Hal ini berarti algoritma ini bertipe in-place. Pendekatan utamanya adalah perbandingan dan pertukaran elemen bersebelahan secara terus menerus hingga seluruh data tidak ada lagi yang salah urutan.

Untuk analisis operasi, dari output tersebut untuk jumlah comparison atau perbandingan bergantung pada bentuk data. Pada data yang sudah terurut ($N = 1000$), komparasi hanya dilakukan sebanyak 999 kali. Namun pada data terurut menurun ($N = 1000$), komparasi membengkak hingga hampir 500.000 kali.

Jumlah pertukaran berjalan konsisten dengan kondisi kekacauan data. Pada data terurut, tidak ada swap sama sekali (0). Pada data terurut menurun, jumlah swap bernilai sama persis dengan jumlah komparasinya, membuktikan bahwa setiap perbandingan berakhir dengan pertukaran elemen.

Jumlah shift dan rekursi bernilai 0, Bubble Sort tetap mengakses elemen array untuk membandingkan dan menukar nilai.

Bentuk data yang rapi memperkecil jumlah komparasi dan swap secara ekstrem, sedangkan data terbalik atau reverse sorted akan memaksa sistem bekerja menyentuh batas maksimum komparasi dan swap.

Untuk performanya, waktu eksekusi algoritma ini kurang efisien untuk jumlah elemen dataset yang besar. Pada data acak $N = 100$, waktu yang dicatat sangat cepat (0,03 ms). Namun ketika jumlah data dikalikan 100 menjadi $N=10000$, waktu eksekusinya melompat tinggi menjadi sekitar 120 ms.

Jika data awalnya sudah rapi, waktu proses sangat instan berkat deteksi awal. Contohnya pada $N = 10000$ untuk data Sorted hanya membutuhkan 0,005 ms. Hal ini sangat berbanding terbalik dengan data Reverse Sorted di jumlah yang sama, yang membutuhkan waktu nyaris 120 ms.

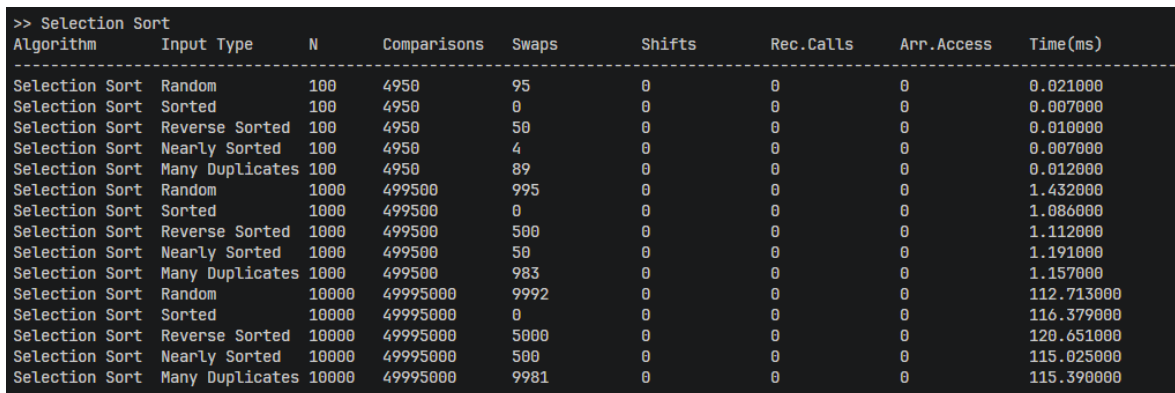
Hasil output membuktikan teori kompleksitas. Secara teoritis, best case adalah $O(n)$, dan output membuktikan bahwa saat $N = 10000$ Sorted, jumlah komparasi adalah $N - 1$ yaitu 9999. Secara teoritis worst case adalah $O(n^2)$, yang terbukti saat ukuran elemen (N)

dinaikkan 10 kali lipat (dari 1000 ke 10000) pada data Reverse Sorted, beban komparasinya juga melonjak drastis, sehingga waktu eksekusinya membesar sekitar 100 kali lipat

SELECTION SORT

Di kode selection sort, terdapat dua perulangan utama. Perulangan luar berfungsi untuk menentukan posisi yang sedang ingin diisi yang dimulai dari indeks paling kiri. Kemudian, perulangan dalam bertugas memindai seluruh sisa elemen di sebelah kanannya untuk mencari mana nilai yang paling kecil (`min_idx`). Setelah pemindaian selesai dan nilai terkecil ditemukan, nilai tersebut ditukar atau swap dengan elemen di posisi target.

Tujuan dari cara kerja ini adalah untuk mengurutkan data dengan cara memilih elemen terkecil dari sisa data yang belum terurut, lalu menempatkannya di ujung kiri zona data yang sudah terurut, secara bertahap.



>> Selection Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Selection Sort	Random	100	4950	95	0	0	0	0.021000
Selection Sort	Sorted	100	4950	0	0	0	0	0.007000
Selection Sort	Reverse Sorted	100	4950	50	0	0	0	0.010000
Selection Sort	Nearly Sorted	100	4950	4	0	0	0	0.007000
Selection Sort	Many Duplicates	100	4950	89	0	0	0	0.012000
Selection Sort	Random	1000	499500	995	0	0	0	1.432000
Selection Sort	Sorted	1000	499500	0	0	0	0	1.086000
Selection Sort	Reverse Sorted	1000	499500	500	0	0	0	1.112000
Selection Sort	Nearly Sorted	1000	499500	50	0	0	0	1.191000
Selection Sort	Many Duplicates	1000	499500	983	0	0	0	1.157000
Selection Sort	Random	10000	49995000	9992	0	0	0	112.713000
Selection Sort	Sorted	10000	49995000	0	0	0	0	116.379000
Selection Sort	Reverse Sorted	10000	49995000	5000	0	0	0	120.651000
Selection Sort	Nearly Sorted	10000	49995000	500	0	0	0	115.025000
Selection Sort	Many Duplicates	10000	49995000	9981	0	0	0	115.390000

Gambar 2 Screenshot Hasil Uji Coba Selection Sort

Berbeda dengan Bubble Sort yang bisa berhenti lebih awal, Selection Sort sangat kaku. Walaupun datanya sudah terurut (Best Case), acak (Average Case), maupun terurut terbalik (Worst Case), perulangan dalam akan tetap memaksa mencari nilai terkecil hingga ke ujung akhir array. Algoritma ini tidak memiliki cara untuk mengetahui apakah sisa datanya sudah rapi atau belum.

Karena setiap elemen pasti dibandingkan dengan sisa elemen lainnya tanpa terkecuali, jumlah operasi perbandingannya selalu sama untuk setiap bentuk data, yaitu sekitar $\frac{n(n-1)}{2}$.

Ini berarti untuk Best Case: $O(n^2)$, Average Case: $O(n^2)$, Worst Case: $O(n^2)$

Algoritma ini melakukan proses pengurutan yaitu pencarian dan pertukaran langsung di dalam array bawaan yang diberikan. Variabel memori tambahan yang digunakan hanyalah variabel pelacak indeks yang sederhana, seperti `i`, `j`, dan `min_idx`. Tidak

membutuhkan pembuatan array kosong baru yang ukurannya memakan memori seiring bertambahnya data. Algoritma ini membutuhkan memori tambahan yang sangat kecil, yaitu $O(1)$. Ini berarti, selection Sort tergolong algoritma In-place.

Ketika Selection Sort menukar nilai terkecil dari area kanan ke posisi kiri, pertukaran ini bisa melompati elemen lain. Misalnya, jika ada angka kembar, elemen kembar yang berada di depan bisa tertukar dan terlempar jauh ke belakang posisinya. Karena urutan asli elemen kembar bisa rusak akibat lompatan ini, kesimpulannya algoritma ini bersifat tidak stabil.

Mekanisme intinya adalah pencarian nilai minimum secara linear pada sisa array, kemudian menukarnya ke posisi yang benar.

Berdasarkan output hasil eksekusi pada tabel $N=100$, $N=1000$, $N=10000$ untuk:

Jumlah Comparison (Perbandingan): Angkanya identik di setiap jenis data. Pada $N = 100$, jumlah komparasi selalu tepat 4950. Pada $N = 1000$ selalu 499500, dan pada $N = 10000$, selalu 49995000. Hal ini membuktikan bahwa algoritma ini selalu melakukan pemindaian penuh terlepas dari bentuk datanya.

Jumlah Swap (Pertukaran): Pada data yang sudah terurut yaitu Sorted, jumlah swap adalah 0. Pada data acak Random, swap bervariasi dan umumnya mendekati jumlah putaran perulangan luar, dengan maksimum sekitar $n - 1$.

Jumlah Shift, Rekursi: Semua bernilai 0 karena algoritma ini murni menggunakan mekanisme tukar posisi atau swap, bukan pergeseran bertahap, dan tidak menggunakan fungsi rekursif.

Pengaruh Bentuk Data: Bentuk/distribusi data tidak mempengaruhi jumlah komparasi, tetapi mempengaruhi jumlah swap.

Untuk performa buat dataset kecil $N = 100$, algoritma ini cukup cepat sekitar 0,02 ms. Namun, karena memiliki kompleksitas kuadratik $O(n^2)$, menurun secara signifikan pada dataset besar. Saat N naik menjadi 10000, waktu eksekusinya meroket tajam menjadi rata-rata 115 ms.

Pada dataset $N = 10000$, waktu eksekusi untuk semua jenis data (Random, Sorted, Reverse Sorted) berada di kisaran yang sama, yaitu antara 112 ms hingga 120 ms. Ini berarti distribusi data hampir tidak memberikan pengaruh signifikan pada kecepatan eksekusi Selection Sort.

Secara teori kompleksitas Selection Sort di semua skenario (Best, Average, Worst) adalah $O(n^2)$. Output menunjukkan jumlah komparasi yang identik pada semua kondisi input data, sehingga mendukung teori bahwa Selection Sort memiliki kompleksitas waktu $O(n^2)$ pada best, average, dan worst case. Waktu eksekusi yang relatif mirip antar jenis data juga sesuai dengan karakteristik algoritma ini.

INSERTION SORT

Algoritma ini membagi array menjadi dua bagian secara imajiner, bagian kiri yang sudah terurut dan bagian kanan yang belum terurut. Algoritma mengambil satu elemen dari bagian yang belum terurut yang disimpan dalam variabel `key`, lalu membandingkannya dengan elemen-elemen di bagian kiri dari belakang ke depan. Jika elemen di kiri lebih besar dari `key`, elemen tersebut digeser ke kanan.

Proses ini berlanjut sampai ditemukan posisi yang tepat untuk menyisipkan elemen tersebut. Mekanismenya mirip saat mengurutkan kartu remi di tangan. Tujuannya adalah menyisipkan setiap elemen satu per satu ke posisi yang benar.

>> Insertion Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Insertion Sort	Random	100	2656	0	2663	0	0	0.006000
Insertion Sort	Sorted	100	99	0	99	0	0	0.000000
Insertion Sort	Reverse Sorted	100	4950	0	5049	0	0	0.020000
Insertion Sort	Nearly Sorted	100	395	0	395	0	0	0.001000
Insertion Sort	Many Duplicates	100	2412	0	2414	0	0	0.005000
Insertion Sort	Random	1000	244181	0	244189	0	0	0.569000
Insertion Sort	Sorted	1000	999	0	999	0	0	0.003000
Insertion Sort	Reverse Sorted	1000	499500	0	500499	0	0	1.911000
Insertion Sort	Nearly Sorted	1000	26775	0	26777	0	0	0.057000
Insertion Sort	Many Duplicates	1000	241801	0	241807	0	0	0.634000
Insertion Sort	Random	10000	25128135	0	25128147	0	0	62.011000
Insertion Sort	Sorted	10000	9999	0	9999	0	0	0.023000
Insertion Sort	Reverse Sorted	10000	49995000	0	50004999	0	0	122.514000
Insertion Sort	Nearly Sorted	10000	3129351	0	3129351	0	0	7.332000
Insertion Sort	Many Duplicates	10000	25103078	0	25103085	0	0	61.832000

Gambar 3 Screenshot Hasil Uji Coba Insertion Sort

Untuk Kompleksitas waktunya, Best Case terjadi jika data yang diberikan sejak awal sudah terurut menaik Sorted. Saat algoritma mengambil `key` dan membandingkannya dengan elemen sebelah kirinya, elemen kirinya pasti lebih kecil. Kondisi pengecekan langsung berhenti saat itu juga, sehingga algoritma hanya melakukan satu kali perbandingan (comparison) pada setiap elemen dan tidak memerlukan pergeseran (swap) elemen dalam jumlah besar. Kompleksitas waktu terbaiknya adalah $O(n)$.

Average Case: Jika datanya acak (Random), setiap elemen rata-rata perlu dibandingkan dan digeser melewati sebagian elemen pada bagian kiri yang sudah terurut sebelum menemukan posisi yang sesuai. Kompleksitas waktu rata-ratanya adalah $O(n^2)$.

Worst Case: Kondisi terburuk terjadi ketika data terurut menurun (Reverse Sorted). Setiap kali elemen `key` diambil, selalu lebih kecil dari semua elemen di sebelah kirinya. Akibatnya,

sistem harus menggeser seluruh elemen tersebut sampai ke indeks ke 0 pada setiap putarannya. Kompleksitas waktu terburuknya adalah $O(n^2)$.

Untuk Kompleksitas Ruang, proses pengurutan dilakukan langsung pada array asli tanpa menggunakan array tambahan yang ukurannya bergantung pada jumlah data. Variabel tambahan yang digunakan hanyalah penampung nilai sementara (*key*) dan indeks penunjuk arah (*i*, *j*). Kompleksitas ruangnya sangat minim, yaitu $O(1)$. Karena tidak memerlukan memori array tambahan yang mengikuti jumlah elemen, Insertion Sort merupakan algoritma In-place.

Algoritma ini hanya akan menggeser elemen di kirinya jika elemen tersebut lebih besar ($data[j] > key$). Jika terdapat elemen dengan nilai yang sama, kondisi $data[j] > key$ akan bernilai false sehingga elemen tidak digeser. Akibatnya, urutan relatif elemen yang bernilai sama tetap terjaga. Ini menjaga urutan asli dari elemen kembar. Ini berarti, algoritma ini bersifat Stable. Mekanisme intinya adalah perbandingan yang dilanjutkan dengan pergeseran (shift) dan penyisipan (insert), bukan pertukaran (swap).

Analisis operasinya berdasarkan output didapat data pada tabel menunjukkan kepekaan algoritma ini. Pada data yang sudah terurut ($N=10000$), algoritma hanya melakukan 9999 perbandingan dan pergeseran, menunjukkan bahwa performanya bersifat linear pada kondisi terbaik. Namun pada data Reverse Sorted ($N = 10000$), jumlah perbandingan dan pergeseran meningkat drastis hingga sekitar 50 juta operasi masing-masing pada kondisi Reverse Sorted dengan $N = 10000$. Jumlah Swap bernilai 0 di semua kondisi. Ini berarti kodenya bekerja secara efisien menimpa elemen secara bergeser ($data[j+1] = data[j]$), algoritma menggunakan mekanisme shifting dibandingkan pertukaran langsung antar elemen, sehingga jumlah swap tercatat 0 pada seluruh pengujian.

Jumlah Recursion Call & Array Accessed: Keduanya bernilai 0 karena algoritma ini bersifat iteratif (menggunakan loop) dan operasi hitungannya difokuskan pada pergeseran.

Insertion Sort termasuk algoritma adaptive karena performanya meningkat signifikan ketika data sudah sebagian terurut (Nearly Sorted). Jumlah operasinya akan turun drastis ke angka yang sangat rendah dibandingkan data acak.

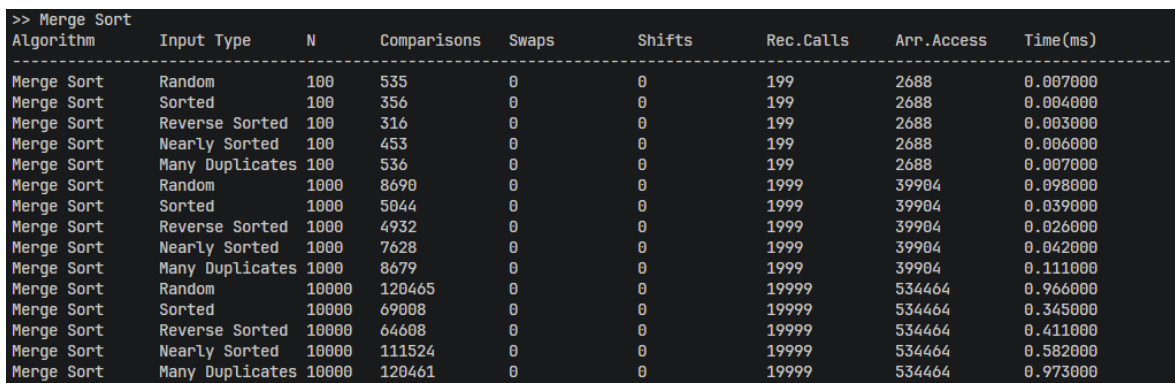
Untuk waktu eksekusi antar data performanya cepat untuk jumlah data kecil. Pada kasus Nearly Sorted ($N = 10000$) selesai hanya dalam kurang lebih 7 ms. Ini jauh lebih kencang dibandingkan Bubble atau Selection Sort. Namun, pada dataset besar yang berantakan atau terbalik, waktunya melonjak tajam menjadi sekitar 122 ms.

Pengaruh Distribusi Data: Di tabel output memperlihatkan bahwa semakin rapi data aslinya, semakin cepat Insertion Sort menyelesaikannya (waktu 0,023 ms untuk data Sorted $N = 10000$ dan 122,5 ms untuk Reverse Sorted). Hasil eksperimen menunjukkan kesesuaian dengan teori kompleksitas waktu Insertion Sort. Teori $O(n)$ pada Best Case dibuktikan dengan jumlah komparasi persis sebesar $N - 1$ (9999 untuk $N = 10000$). Sedangkan teori $O(n^2)$ terbukti dengan jumlah operasi meningkat sangat besar seiring bertambahnya ukuran data ketika menemui data terbalik.

MERGE SORT

Algoritma ini membagi array terus-menerus menjadi dua bagian di titik tengah (*mid*) secara rekursif sampai setiap bagian hanya tersisa satu elemen. Setelah array terbagi menjadi bagian-bagian kecil berisi satu elemen, fungsi *do_merge* akan menyatukannya atau menggabungkan kembali bagian-bagian tersebut secara bertahap. Saat proses penyatuan ini, elemen dari kepingan kiri dan kanan dibandingkan, lalu elemen dengan nilai lebih kecil dipindahkan ke array sementara (*temp*) secara berurutan. Setelah array sementara rapi, isinya disalin kembali ke array aslinya.

Tujuan dari pendekatan ini adalah mempercepat proses pengurutan dengan prinsip pecah dan taklukkan (*divide and conquer*). Daripada mengurutkan satu data besar, algoritma ini mengurutkan bagian-bagian kecil yang jauh lebih ringan, lalu menggabungkannya kembali menjadi satu array utuh yang terurut.



>> Merge Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Merge Sort	Random	100	535	0	0	199	2688	0.007000
Merge Sort	Sorted	100	356	0	0	199	2688	0.004000
Merge Sort	Reverse Sorted	100	316	0	0	199	2688	0.003000
Merge Sort	Nearly Sorted	100	453	0	0	199	2688	0.006000
Merge Sort	Many Duplicates	100	536	0	0	199	2688	0.007000
Merge Sort	Random	1000	8690	0	0	1999	39904	0.098000
Merge Sort	Sorted	1000	5044	0	0	1999	39904	0.039000
Merge Sort	Reverse Sorted	1000	4932	0	0	1999	39904	0.026000
Merge Sort	Nearly Sorted	1000	7628	0	0	1999	39904	0.042000
Merge Sort	Many Duplicates	1000	8679	0	0	1999	39904	0.111000
Merge Sort	Random	10000	120465	0	0	19999	534464	0.966000
Merge Sort	Sorted	10000	69008	0	0	19999	534464	0.345000
Merge Sort	Reverse Sorted	10000	64608	0	0	19999	534464	0.411000
Merge Sort	Nearly Sorted	10000	111524	0	0	19999	534464	0.582000
Merge Sort	Many Duplicates	10000	120461	0	0	19999	534464	0.973000

Gambar 4 Screenshot Hasil Uji Coba Merge Sort

Untuk Kompleksitas Waktu di Best Case, Average Case, dan Worst Case adalah $O(n \log n)$, karena proses membelah array akan selalu menghasilkan tingkatan pohon logaritmik ($\log n$), dan proses menggabungkannya kembali akan selalu menyentuh seluruh elemen sebanyak n kali pada setiap tingkatan tersebut, sebagian besar proses algoritma bersifat tetap.

Program tidak peduli apakah data awalnya sudah terurut rapi, acak acakan, atau terbalik, ia tetap akan melakukan proses pembelahan dan penggabungan rekursif dengan pola kerja yang relatif tetap pada setiap kondisi data. Kompleksitas waktu untuk semua skenario (Terbaik, Rata-rata, Terburuk) sama yaitu $O(n \log n)$

Pada Kompleksitas Ruang, karena untuk menggabungkan dua sub-array dengan aman tanpa menimpa data aslinya program membutuhkan sebuah array penampung sementara seperti yang terlihat pada baris `std::vector<int> temp(data.size());`; di dalam kode merge sort, ukuran array sementara ini akan sama persis dengan ukuran data aslinya (n). Artinya, algoritma ini memakan memori tambahan yang signifikan yaitu $O(n)$. Oleh sebab itu, algoritma ini tidak bisa menghemat tempat dan tergolong sebagai algoritma Not In-place.

Pada aturan penggabungannya (`if (data[i] <= data[j])`), jika nilai elemen dari kepingan kiri sama dengan elemen dari kepingan kanan, elemen dari sub-array kiri diproses lebih dahulu ketika nilainya sama karena kondisi menggunakan operator `<=`, sehingga urutan relatif elemen yang sama tetap terjaga di array sementara (`temp`). Hal ini membuat elemen kembar yang sejak awal ada di depan tidak akan pernah tersalip oleh kembarannya yang ada di belakang yang mana ini berarti algoritma bersifat Stable.

Cara utamanya adalah membelah masalah menjadi sub-masalah kecil lalu menyelesaikannya dari bawah ke atas, ini adalah Divide and Conquer.

Jumlah Recursion Call (Pemanggilan Rekursif): karena program mutlak membelah data hingga tersisa satu elemen tanpa syarat berhenti di tengah jalan, jumlah pemanggilan rekursif bersifat tetap untuk setiap ukuran data. Untuk $N = 100$ nilainya 199, dan untuk $N = 10000$ nilainya 19999, ini berarti selalu memiliki total rekursi sebesar $2n - 1$ untuk kondisi data apa pun.

Jumlah Array Accessed: Proses penyalinan data ke array sementara (`temp`) dan kembali ke array utama mutlak dilakukan setiap saat, nilainya sangat kaku tidak terpengaruh bentuk data (bernilai tetap di sekitar 534.464 akses pada $N = 10000$).

Jumlah Comparison (Perbandingan): Karena saat menggabungkan dua sub-array, proses perbandingan berhenti lebih cepat jika salah satu sub-array sudah habis. Ini sering terjadi pada data yang sudah terurut, sehingga jumlah perbandingannya sedikit bervariasi. Pada $N = 10000$, data Acak (*Random*) butuh sekitar 120.000 komparasi, sedangkan data Terurut (*Sorted*) hanya butuh sekitar 69.000 komparasi.

Jumlah Swap & Shift: Karena proses intinya murni memindahkan data ke array cadangan, tidak ada satupun elemen yang ditukar langsung di tempat (swap) atau digeser beruntun (shift).

Pengaruh Bentuk Data: Bentuk data hanya sedikit meringankan jumlah perbandingan, namun sama sekali tidak bisa meringankan beban rekursi dan akses memori array.

Perbandingan Waktu Eksekusi (Data Kecil vs Besar): Karena menggunakan metode pembelahan logaritmik (divide and conquer), untuk data kecil ($N = 100$) Merge Sort tidak terasa lebih cepat. Namun, untuk data besar ($N = 10.000$), karena Merge Sort terhindar dari jebakan perulangan kuadratik, performanya menjadi jauh lebih efisien dibanding algoritma berkompleksitas kuadratik. Waktu eksekusinya maksimal hanya sekitar 0,9 ms, dibandingkan Bubble Sort yang mencapai 120 ms.

Pengaruh Distribusi Data: Karena porsi terbesar dari algoritma ini (rekursi dan baca/tulis memori) tidak terpengaruh oleh keacakan data, rentang waktu antar tipe data sangat tipis. Performa Merge Sort relatif stabil terhadap berbagai distribusi data.

Tabel menunjukkan waktu komputasi yang tidak melonjak drastis menjadi ratusan milidetik, baik pada Best Case maupun Worst Case. Hasil eksperimen ini membuktikan bahwa kompleksitas waktu Merge Sort tetap $O(n \log n)$ pada seluruh kondisi data.

QUICK SORT

Pada fungsi `do_quick_sort`, algoritma ini bekerja dengan memilih sebuah elemen sebagai nilai pivot. Nilai yang dijadikan patokan dipilih dari elemen pada indeks tengah array (baris `int pivot = data[mid]`). Setelah itu, dua buah penunjuk (`i` dari ujung kiri dan `j` dari ujung kanan) bergerak ke tengah.

Penunjuk kiri bergerak ke kanan selama elemen yang ditemukan masih lebih kecil dari pivot, sedangkan penunjuk kanan bergerak ke kiri selama elemennya lebih besar dari pivot. Ketika kedua penunjuk berhenti pada elemen yang berada di sisi yang salah terhadap pivot, posisinya ditukar (*swap*). Proses ini berlanjut sampai semua elemen yang lebih kecil dari pivot berada di sebelah kiri, dan yang lebih besar berada di kanan.

Setelah berhasil dipisah, program memanggil dirinya sendiri (rekursi) untuk memecah dan mengurutkan sub-array kiri dan sub-array kanan secara mandiri.

Tujuannya adalah mengurutkan data dengan prinsip Divide and Conquer (pecah dan taklukkan). Mempartisi data besar menjadi sub-array yang lebih kecil sehingga proses pengurutan dapat dilakukan lebih cepat dibandingkan algoritma berkompleksitas kuadratik pada rata-rata kasus.

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Quick Sort	Random	100	807	189	0	171	0	0.007000
Quick Sort	Sorted	100	606	63	0	127	0	0.002000
Quick Sort	Reverse Sorted	100	610	112	0	127	0	0.003000
Quick Sort	Nearly Sorted	100	606	67	0	127	0	0.003000
Quick Sort	Many Duplicates	100	720	236	0	153	0	0.007000
Quick Sort	Random	1000	13577	2571	0	1775	0	0.078000
Quick Sort	Sorted	1000	9009	511	0	1023	0	0.016000
Quick Sort	Reverse Sorted	1000	9016	1010	0	1023	0	0.020000
Quick Sort	Nearly Sorted	1000	9231	698	0	1101	0	0.024000
Quick Sort	Many Duplicates	1000	11713	3055	0	1527	0	0.061000
Quick Sort	Random	10000	180228	33566	0	17781	0	0.929000
Quick Sort	Sorted	10000	125439	5904	0	11809	0	0.193000
Quick Sort	Reverse Sorted	10000	125452	10904	0	11811	0	0.193000
Quick Sort	Nearly Sorted	10000	126947	8064	0	11771	0	0.330000
Quick Sort	Many Duplicates	10000	161287	38673	0	15473	0	0.792000

Gambar 5 Screenshot Hasil Uji Coba Quick Sort

Untuk kompleksitas waktu algoritma quick sort, Best Case terjadi apabila nilai pivot yang terpilih selalu berhasil membelah array tepat menjadi dua bagian yang sama besar. Karena pivot dipilih dari elemen indeks tengah (`data[mid]`), maka data yang sudah terurut cenderung menghasilkan pembagian sub-array yang relatif seimbang. Kompleksitas waktu terbaiknya adalah $O(n \log n)$.

Average Case terjadi ketika data acak membuat pivot tidak selalu membagi tepat di tengah, namun secara rata-rata kedalaman pembelahannya tetap proporsional. Kompleksitas waktu rata-ratanya adalah $O(n \log n)$.

Worst Case terjadi ketika pivot terus-menerus menghasilkan pembagian sub-array yang tidak seimbang, sehingga kedalaman rekursi mendekati n . Pada kondisi tersebut kompleksitas waktunya dapat membesar menjadi $O(n^2)$.

Pada kompleksitas ruang, algoritma ini menukar elemen langsung di dalam array aslinya. Akan tetapi, setiap kali ia memanggil fungsi rekursif, sistem operasi butuh menyimpan memori riwayat pemanggilan di latar belakang yang dikenal dengan Call Stack. Kedalaman tumpukan memori ini proporsional dengan tinggi pohon pembagian data, yaitu $\log n$.

Pada kondisi rata rata, kedalaman rekursi Quick Sort bersifat logaritmik sehingga kompleksitas ruang tambahannya adalah $O(\log n)$. Namun pada worst case, kedalaman rekursi dapat membesar hingga $O(n)$.

Karena hanya membutuhkan memori tambahan untuk call stack rekursi dan tidak membuat array cadangan sebesar data asli, algoritma ini tergolong In-place.

Algoritma ini tidak bersifat stable karena proses swap dapat mengubah urutan relatif elemen yang memiliki nilai sama. Urutan historis elemen kembar tidak dijamin utuh. Mekanisme utamanya adalah pemilihan pivot dan proses partisi dengan pendekatan divide and conquer.

Untuk Jumlah Comparison dan Swap, karena partisi data acak butuh banyak pergerakan untuk dikelompokkan, nilai komparasinya tertinggi di Random (180.228 perbandingan di $N = 10.000$). Pada data terurut (Sorted dan Reverse Sorted), operasi menurun jauh ke kisaran 125.000. Hal ini menunjukkan bahwa pemilihan pivot dari indeks tengah menghasilkan pembagian sub-array yang relatif seimbang.

Jantung algoritma ini adalah rekursi, jumlah pemanggilan rekursif berada pada kisaran 11.000 hingga 17.000 kali untuk $N = 10.000$. Jumlah Shift dan Array Accessed selalu 0, karena implementasi melakukan pertukaran elemen menggunakan fungsi `swap`, bukan menggesernya secara sepihak, dan metrik akses memori tidak dihitung pada algoritma komparasi ini.

Bentuk data terurut meringankan beban perbandingan dan jumlah rekursi, karena pivot dari indeks tengah menghasilkan partisi yang lebih seimbang pada data terurut.

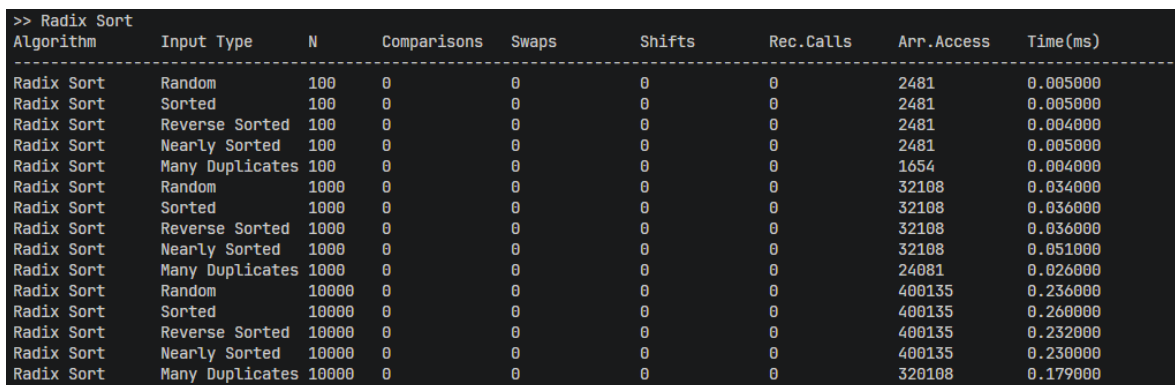
Algoritma ini memiliki efisiensi yang bagus. Untuk himpunan data kecil ($N = 100$), eksekusinya kurang dari 0,01 ms. Untuk data besar ($N = 10.000$), algoritma ini selesai dalam hitungan pecahan milidetik (maksimal hanya 0,929 ms). Untuk data terurut, waktunya paling cepat 0,193 ms, sedangkan data acak butuh usaha lebih ekstra sehingga sedikit lebih lambat (0,929 ms) namun tetap memiliki performa yang sangat efisien.

Secara teori, Quick Sort dapat mencapai worst case $O(n^2)$ bila pivot buruk. Namun, pada implementasi ini pivot diambil dari elemen tengah, sehingga data sorted tidak lagi menjadi kasus buruk dan hasil eksperimen menunjukkan bahwa strategi pemilihan pivot dari indeks tengah berhasil menjaga performa tetap mendekati $O(n \log n)$ pada seluruh distribusi data yang diuji.

RADIX SORT

Pada fungsi `radix_sort` dan `count_sort_for_radix` di algoritma ini proses pengurutan utama tidak menggunakan perbandingan langsung antar elemen seperti pada algoritma `comparison sort`. Program memulainya dengan mencari nilai terbesar di dalam kumpulan data untuk mengetahui berapa banyak digit angka maksimal yang harus diproses. Setelah itu, program memproses angka dari digit paling kanan (satuan), lalu puluhan, ratusan, dan seterusnya menggunakan bantuan algoritma `Counting Sort`. Angka-angka tersebut dikelompokkan ke dalam bucket berdasarkan nilai digitnya, berindeks 0 hingga 9 sesuai digit pada posisi yang sedang diproses.

Tujuannya adalah untuk mengurutkan kumpulan data menggunakan pendekatan representasi digit dari setiap angka sebagai dasar pengelompokan data, sehingga ini masuk ke dalam kategori `non-comparison sort` karena proses pengurutan utamanya tidak bergantung pada perbandingan antar elemen.



Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Radix Sort	Random	100	0	0	0	0	2481	0.005000
Radix Sort	Sorted	100	0	0	0	0	2481	0.005000
Radix Sort	Reverse Sorted	100	0	0	0	0	2481	0.004000
Radix Sort	Nearly Sorted	100	0	0	0	0	2481	0.005000
Radix Sort	Many Duplicates	100	0	0	0	0	1654	0.004000
Radix Sort	Random	1000	0	0	0	0	32108	0.034000
Radix Sort	Sorted	1000	0	0	0	0	32108	0.036000
Radix Sort	Reverse Sorted	1000	0	0	0	0	32108	0.036000
Radix Sort	Nearly Sorted	1000	0	0	0	0	32108	0.051000
Radix Sort	Many Duplicates	1000	0	0	0	0	24081	0.026000
Radix Sort	Random	10000	0	0	0	0	400135	0.236000
Radix Sort	Sorted	10000	0	0	0	0	400135	0.260000
Radix Sort	Reverse Sorted	10000	0	0	0	0	400135	0.232000
Radix Sort	Nearly Sorted	10000	0	0	0	0	400135	0.230000
Radix Sort	Many Duplicates	10000	0	0	0	0	320108	0.179000

Gambar 6 Screenshot Hasil Uji Coba Radix Sort

Untuk kompleksitas waktu pada Best Case, Average Case, dan Worst Case, proses dikendalikan oleh jumlah digit angka terbesar yang disebut k dan jumlah keseluruhan data yakni n , program akan memanggil `Counting Sort` sebanyak k kali sesuai jumlah digit maksimum. Setiap panggilannya akan memindai keseluruhan n elemen tanpa peduli apakah data aslinya sudah rapi, acak, atau terbalik. Tidak ada jalan pintas untuk berhenti lebih awal. Kompleksitas waktu untuk semua skenario (Best Case, Average Case, dan Worst Case) adalah sama, yaitu $O(nk)$.

Program selalu mendeklarasikan variabel baru bernama `std::vector<int> output(n)` pada setiap proses digitnya. Ini berarti program secara eksplisit menuntut ketersediaan memori tambahan sebesar jumlah data aslinya untuk menyusun ulang angka angka sementara, ini berarti membutuhkan memori tambahan sebesar $O(n)$ untuk array sementara (`output`). Karena butuh ruang baru berukuran penuh, algoritma ini tergolong sebagai algoritma Not In-place.

Pada bagian akhir proses Counting Sort, penempatan data ke dalam array `output` diproses menggunakan perulangan menurun dari belakang ke depan (`for (int i = n - 1; i >= 0; i--)`). Logika ini menjamin elemen yang muncul lebih dahulu pada digit yang sama akan mempertahankan urutan relatifnya setelah proses penyusunan ke array `output`. Dengan demikian, algoritma ini memelihara urutan historis data kembar, sehingga bersifat Stable.

Mekanisme utamanya tidak melakukan pertukaran (`swap`) dan perbandingan, melainkan mendistribusikan data ke dalam bucket berbasis nilai digit dari satuan hingga digit terbesar.

Pada jumlah Comparison, Swap, Shift, dan Rec.Calls, karena algoritma ini murni memisahkan digit ke dalam bucket tanpa pernah membandingkan elemen satu dan lainnya, metrik metrik tersebut mutlak bernilai 0.

Metrik ini adalah inti dari beban kerja algoritma. Program murni beroperasi dengan membaca nilai data dan menuliskannya ke bucket array sementara. Pada data $N = 10000$, jumlah akses array secara konsisten mencapai sekitar 400.135 akses array pada $N = 10000$.

Bentuk data yang terurut menaik, terbalik, maupun acak sama sekali tidak mengubah jumlah akses array. Namun pada baris data Many Duplicates pada $N = 10000$, jumlah akses array nya menurun drastis menjadi 320.108 karena pada data Many Duplicates, angka maksimal yang dihasilkan (`max_val`) jauh lebih kecil (jumlah digit k lebih sedikit). Karena k mengecil, perulangan kerjanya berkurang, sehingga akses memorinya pun otomatis ikut menurun.

Untuk dataset $N = 10000$, waktunya hanya berkisar di 0,23 ms, jauh lebih cepat dibanding Bubble, Selection, dan Insertion Sort pada dataset besar.

Karena jumlah langkah kerjanya dikunci oleh besaran N dan besaran maksimal digit k , kecepatan waktu eksekusinya relatif stabil dan hanya sedikit dipengaruhi oleh distribusi data.

Teori menyatakan kompleksitasnya sangat dipengaruhi oleh jumlah digit, yaitu $O(nk)$. Eksperimen membuktikan teori ini dengan mencatatkan penurunan waktu paling cepat (menjadi 0,179 ms) pada kasus uji coba Many Duplicates di mana nilai k (panjang digit) menciut lebih pendek dibanding pengujian lainnya

TAUTAN GIT

<https://github.com/DSA26-ULM/module-4-sorting-RyuuZev>